

From Hand-Tuning to Architecture Search

CNN design, transfer learning and zero-cost NAS for satellite land-use classification (EuroSAT)

Emilio Yared · Joe Missisian — Group 7
Master 2 Artificial Intelligence and Data Engineering
AIDE500 — Programming for AI

Problem and dataset

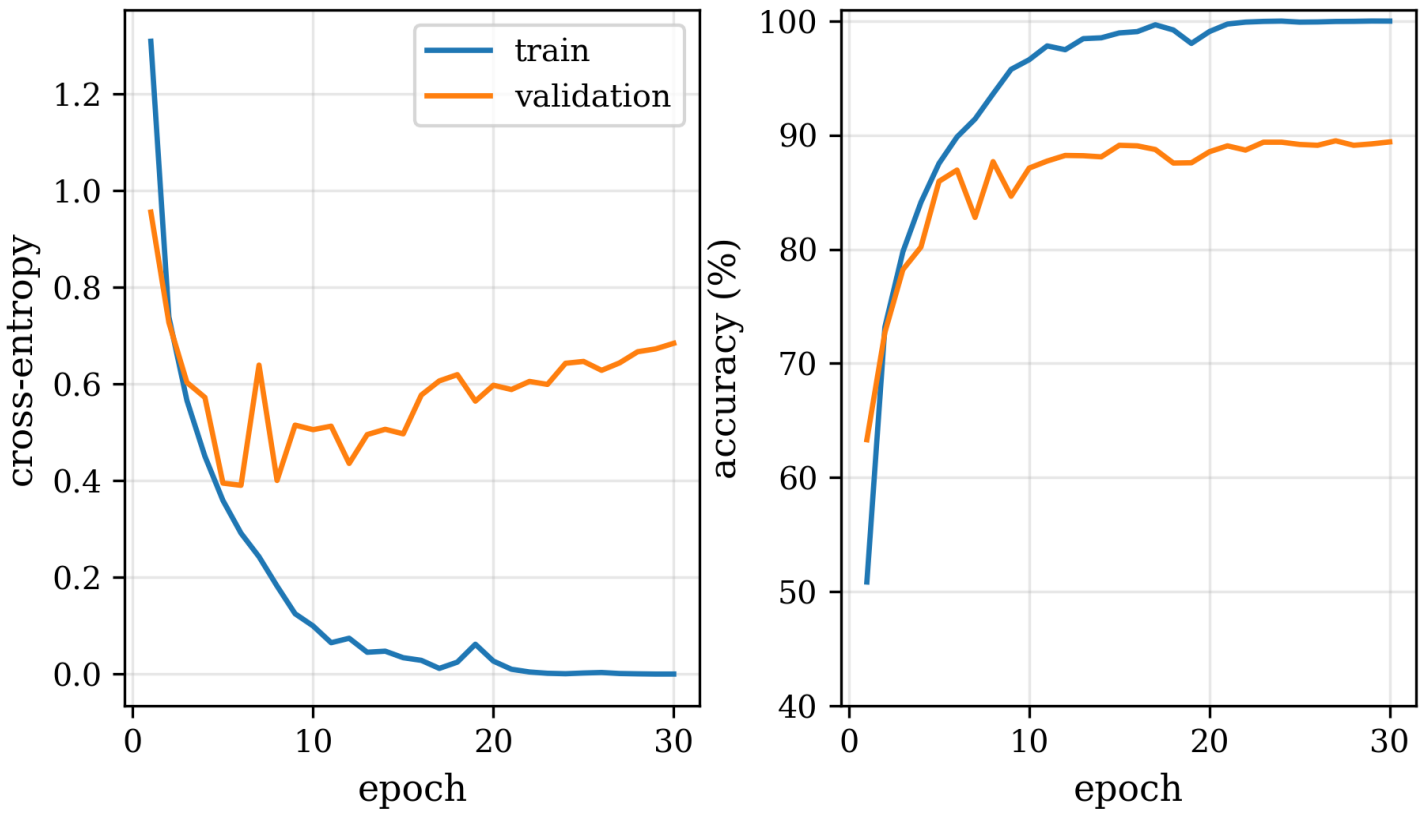


- **EuroSAT RGB**: 27,000 Sentinel-2 tiles, 64×64 px, 10 land-use classes
- Fixed **stratified split** 70/15/15 (18,900 / 4,050 / 4,050), normalization from the train split only
- Classes defined by **texture and color**; several are visually close (crops, vegetation, pasture)

Three questions

1. How far can a CNN trained from scratch go? (Tasks 1–2)
2. What does ImageNet pretraining bring? (Task 3)
3. Can the manual design be automated cheaply? (zero-cost NAS)

Task 1 — Baseline CNN



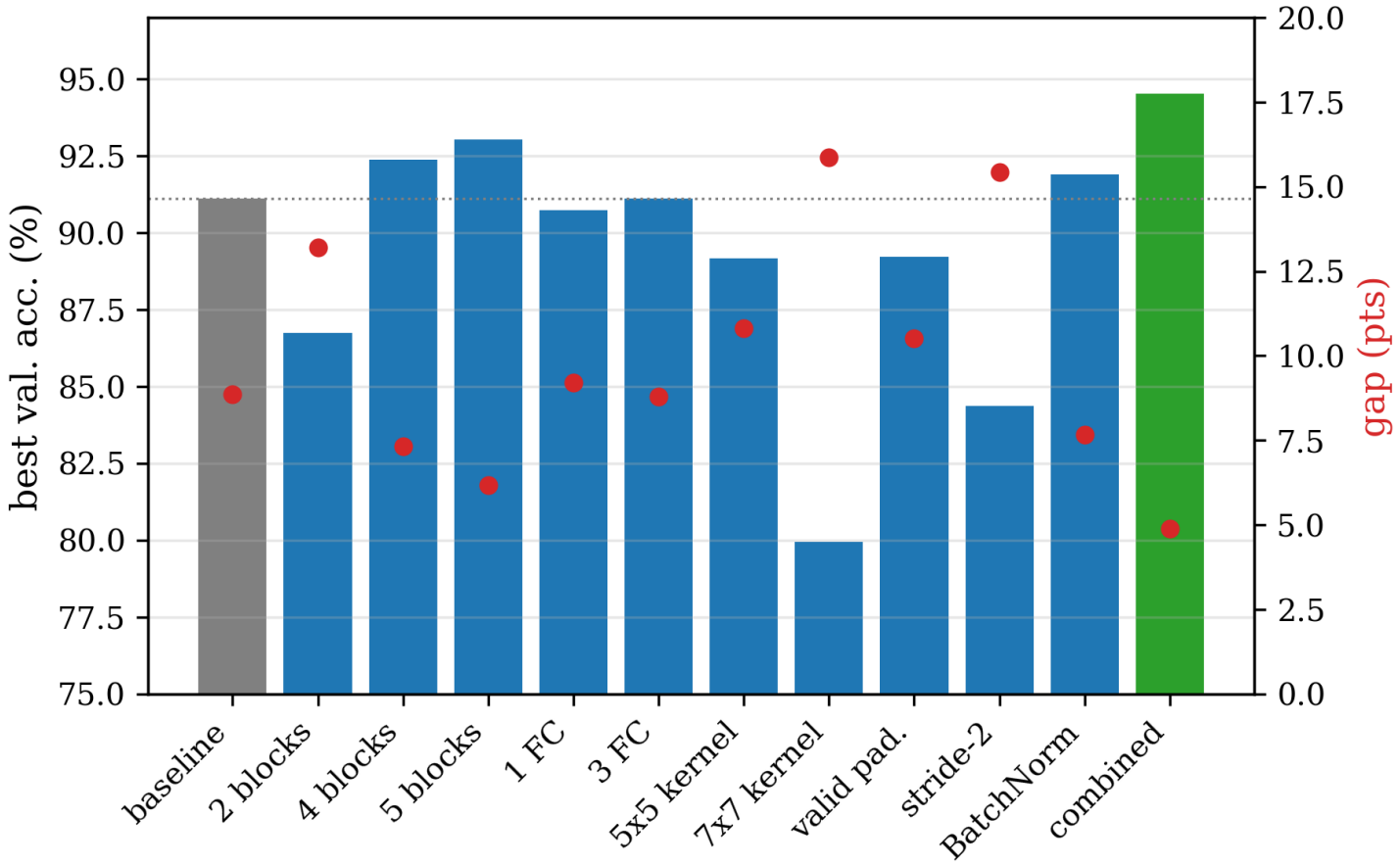
90.15%
test accuracy

10.5 pts
generalization gap

- 3 × Conv3×3 → ReLU → MaxPool (32/64/128 ch.), FC 8192 → 256 → 10, **2.19M** params
- SGD momentum 0.9, lr 0.01, batch 64, 30 epochs
- Main confusions: Highway → River, PermanentCrop → HerbaceousVeg.

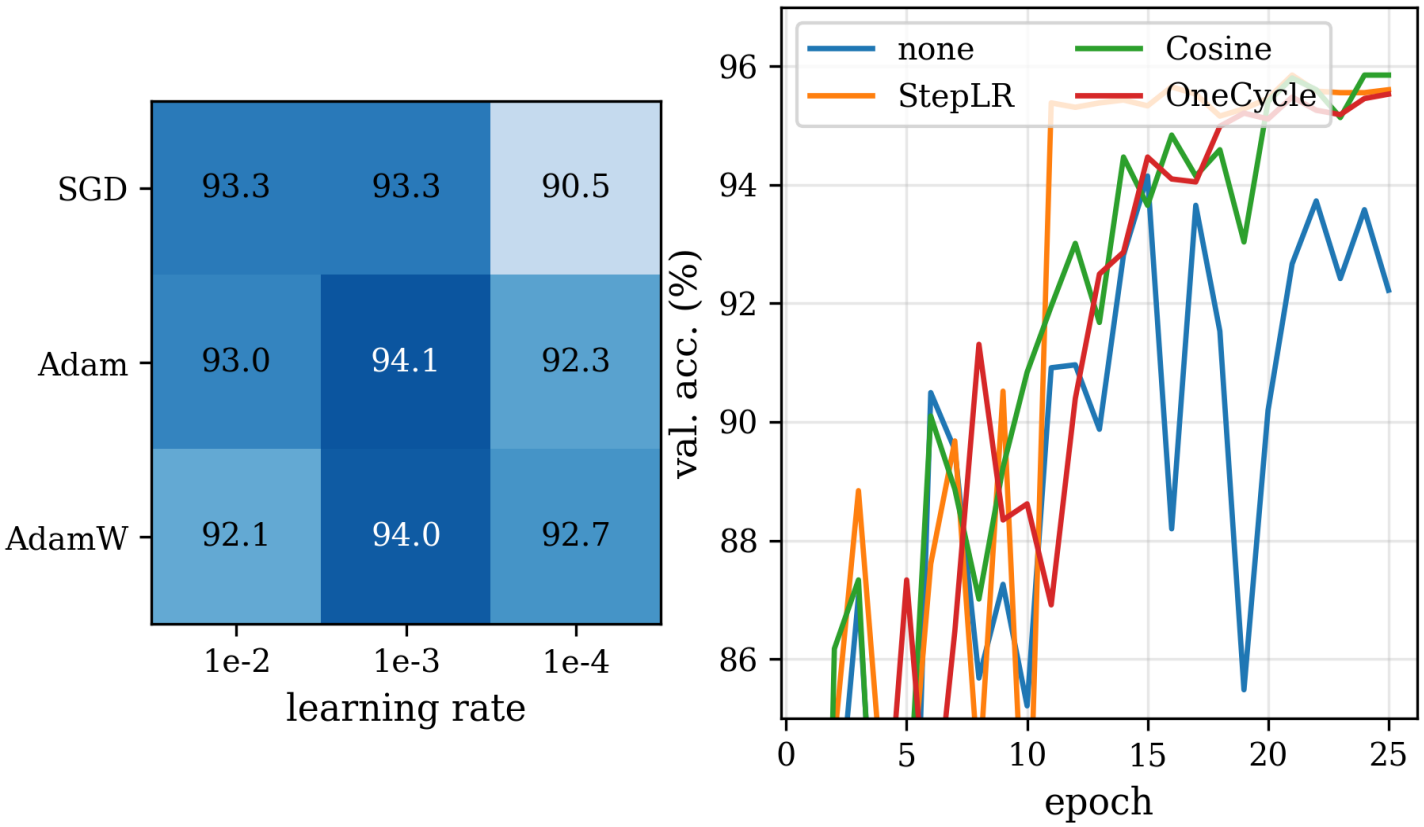
*Train loss keeps falling while the validation loss is minimal at epoch 6: the network **memorizes** the training set.*

Task 2a — Architecture, one factor at a time



- **Depth matters most**: 5 blocks → receptive field 94 px (the whole tile) and **fewer** params (1.24M vs 2.19M)
- **Large kernels hurt**: 7×7 → 79.95%
- Valid padding and stride-2 lose information
- BatchNorm: +0.8 pt, smoother training
- **Combined** (5 blocks, 3×3, same padding, max-pool, BN): **94.52% val.**, gap halved

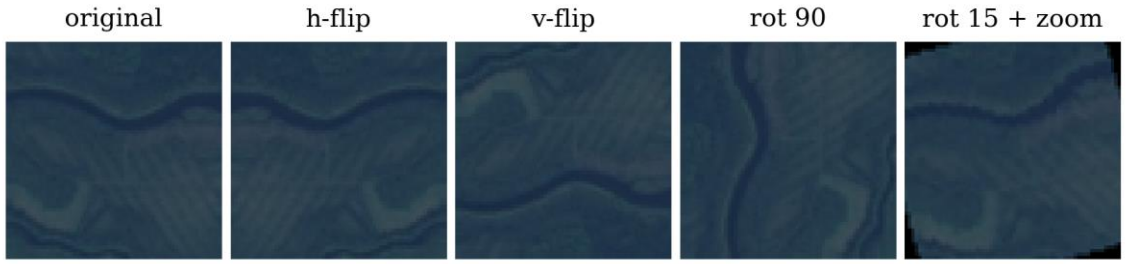
Task 2b — Optimizers, learning rates, schedulers



- 3 optimizers × 3 learning rates, 25 epochs
- **lr 1e-3 best for all**; 1e-2 unstable for Adam/AdamW, 1e-4 too slow for SGD
- Adam 1e-3: best and fastest (94.15%)
- Schedulers add **+1.4–1.7 pts**: small steps at the end remove the oscillations
- Recipe Adam + StepLR: **96.22% test**

Left: peak validation accuracy (%) per optimizer and learning rate. Right: Adam lr 1e-3 with each scheduler.

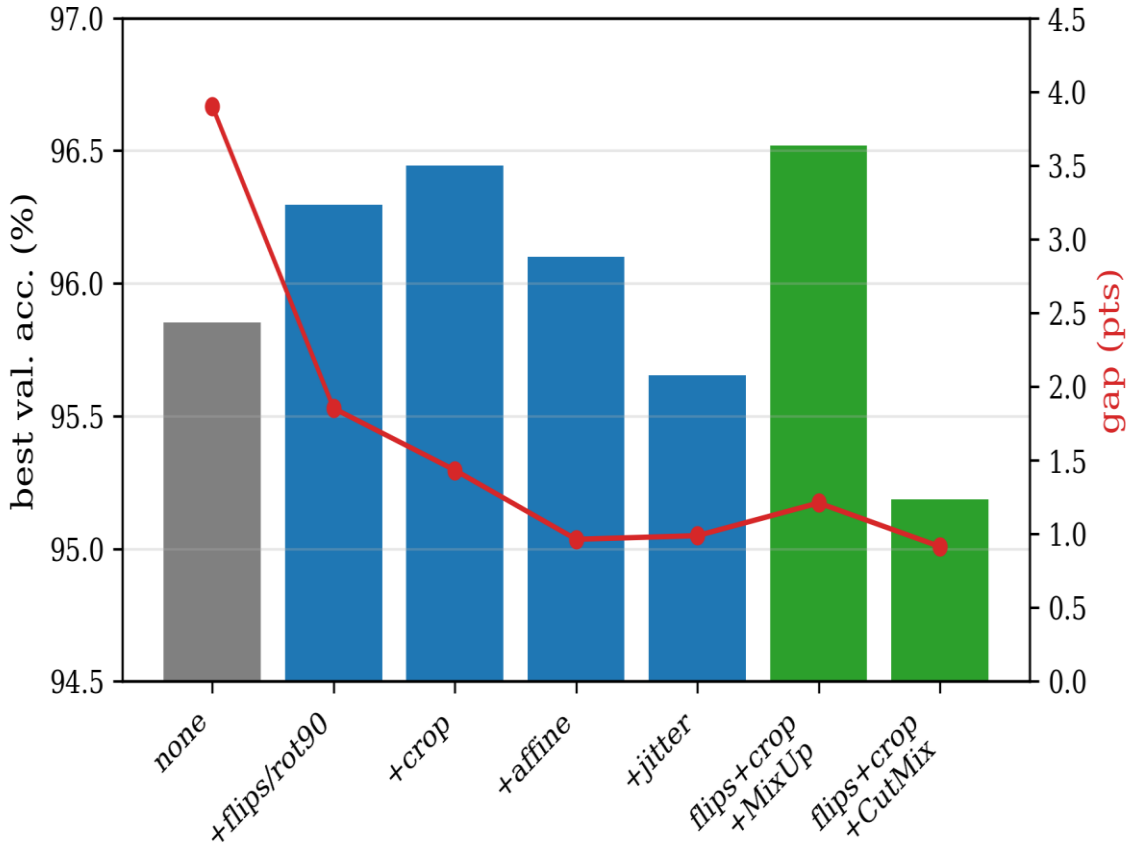
Task 2c — Data augmentation



- No "up" in satellite images: **vertical flips and 90° rotations keep the label**
- Color carries the class → weak color jitter only
- MixUp / CutMix written by hand:

```
loss = lam*CE(out, y_a) + (1-lam)*CE(out, y_b)
```

Gap measured on **clean** training images: 3.9 → ≈1 pt. Affine creates black corners; CutMix needs longer training.



Best: flips + crop + MixUp → **97.04% test**

Task 2 — Comparison table (selection)

NB	Configuration	Params	Latency (ms)	Peak val. (%)	Gap (pts)
NB2	baseline	2.19M	0.380	91.11	8.87
NB2	2 blocks	4.22M	0.302	86.74	13.21
NB2	5 blocks	1.24M	0.635	93.04	6.17
NB2	7×7 kernels	2.61M	0.771	79.95	15.87
NB2	stride-2	2.19M	0.321	84.37	15.43
NB2	BatchNorm	2.19M	0.523	91.90	7.68
NB2	**combined**	**1.24M**	**0.735**	**94.52**	**4.89**

NB	Configuration	Params	Latency (ms)	Peak val. (%)	Gap (pts)
NB3	SGD lr 1e-2	1.24M	0.694	93.28	5.49
NB3	Adam lr 1e-3	1.24M	0.694	94.15	2.47
NB3	Adam + StepLR	1.24M	0.694	95.85	3.31
NB4	no augmentation	1.24M	0.694	95.85	3.90
NB4	flips + crop	1.24M	0.694	96.44	1.43
NB4	**+ MixUp**	**1.24M**	**0.694**	**96.52**	**1.21**
NB4	+ CutMix	1.24M	0.694	95.19	0.91

Latency: one image on a T4 GPU. Gap: train − validation accuracy at the best epoch (NB4: clean training images). The full table (31 runs) is in the report.

90.15 → 95.36%

architecture (NB2)

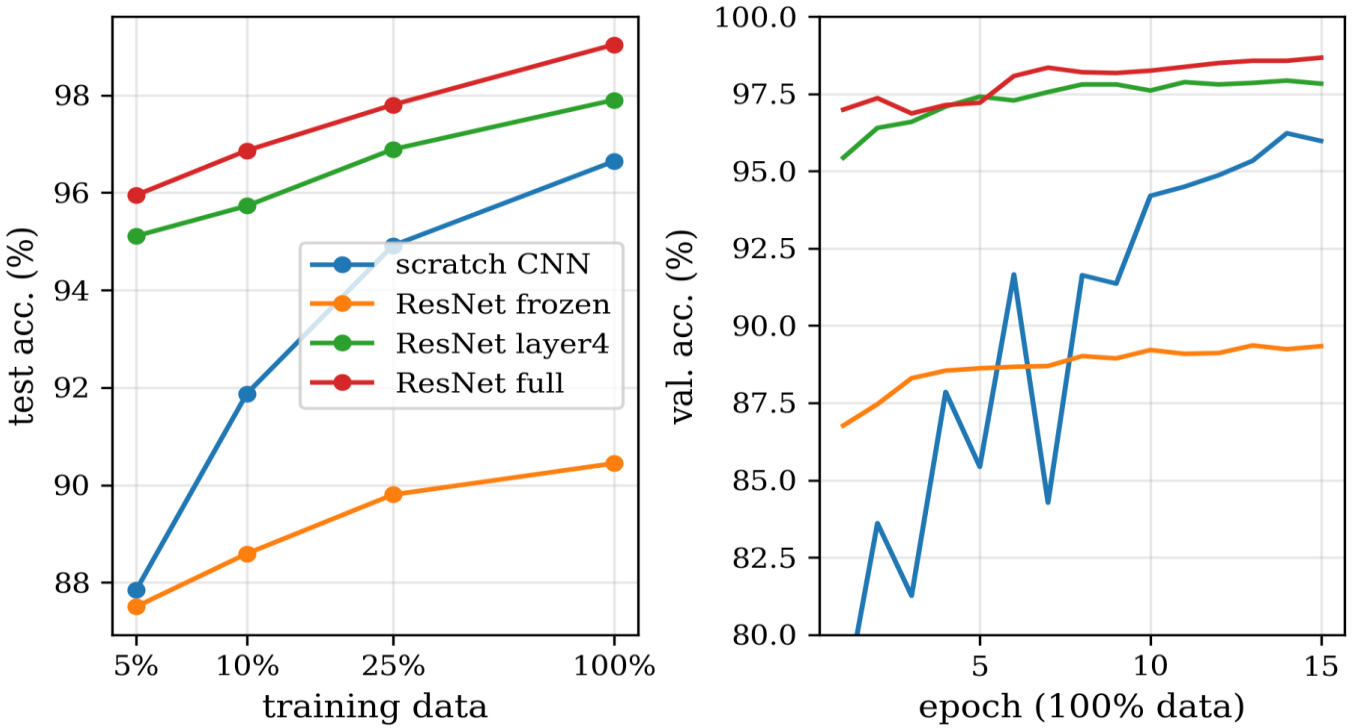
96.22%

+ Adam & StepLR (NB3)

97.04%

+ augmentation & MixUp (NB4)

Task 3 — Transfer learning with ResNet-18



- ImageNet ResNet-18, images resized to 128×128
- Freeze with requires_grad = False; new head Dropout → Linear(512, 10)
- lr 1e-4 for pretrained layers, 1e-3 for the head
- Full fine-tuning: **99.04%**, and 95.95% with **5% of the data** (945 images)
- Converges in 1 epoch (scratch: 6)
- Frozen saturates at ≈90%: unfreezing layer4 recovers most of the gain
- Cost: 9× params, 3× latency

Test acc. (%)	5%	10%	25%	100%
Scratch CNN	87.85	91.88	94.91	96.64
ResNet frozen	87.51	88.59	89.80	90.44
ResNet layer4	95.11	95.73	96.89	97.90
ResNet full	**95.95**	**96.86**	**97.80**	**99.04**

Zero-cost NAS — NASWOT (Mellor et al., ICML 2021)



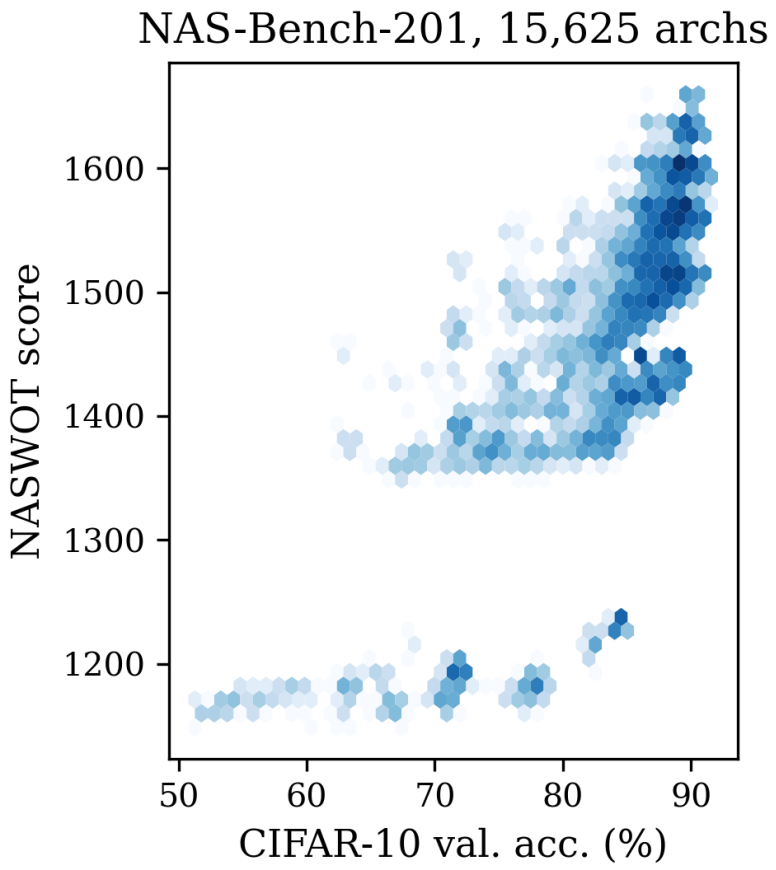
- Judge an architecture **at initialization**, on **one mini-batch**, in ≈0.1 s — no training
- Binary code c_i of each image: which ReLUs are active
- Same code = same linear region → the network cannot separate the two images
- High score = images well separated → the network trains well
- Implemented with **forward hooks** on every nn.ReLU
- Compared with SynFlow, SNIP, grad-norm and #params

$$K_H[i, j] = N_A - d_H(c_i, c_j)$$
$$score \;\; s = \log |K_H|$$

```
def naswot_score(net, inputs):
    B = inputs.size(0)
    K = torch.zeros(B, B, dtype=torch.float64)
    def hook(module, inp, out):
        x = (inp[0].view(B, -1) > 0).double()
        K.add_(x @ x.t() + (1-x) @ (1-x).t())
    for m in net.modules():
        if isinstance(m, nn.ReLU):
            m.register_forward_hook(hook)
    with torch.no_grad():
        net(inputs)
    return torch.linalg.slogdet(K)[1]
```

Simplified from NB6.1

Step 1 — Reproducing the paper on NAS-Bench-201



15,625 networks with **known** trained accuracy. Our re-implementation has **exactly the official number of parameters** for 300/300 architectures.

Kendall τ	CIFAR-10	CIFAR-100	ImageNet16-120*
ours	**0.602**	**0.640**	**0.600**
paper	0.574	0.611	0.549

Search, test acc. (%)	CIFAR-10	CIFAR-100	ImageNet16-120*
NASWOT (N=10) — ours	**92.39 ± 1.20**	**68.63 ± 2.31**	**41.29 ± 3.94**
NASWOT (N=10) — paper	92.44 ± 1.13	68.62 ± 2.04	41.31 ± 4.11
NASWOT (N=100) — ours	**92.83 ± 0.94**	**69.52 ± 1.67**	**43.18 ± 2.95**
NASWOT (N=100) — paper	92.81 ± 0.99	69.48 ± 1.70	43.10 ± 3.16
Random — ours	**87.04 ± 12.94**	**61.36 ± 12.17**	**33.54 ± 9.52**
Random — paper	86.61 ± 13.46	60.83 ± 12.58	33.13 ± 9.66

NASWOT search matches Table 2 of the paper within ≈0.1 pt → **our implementation is correct**

** ImageNet16-120 is not on Kaggle: scored with CIFAR-100 images at 16×16.*

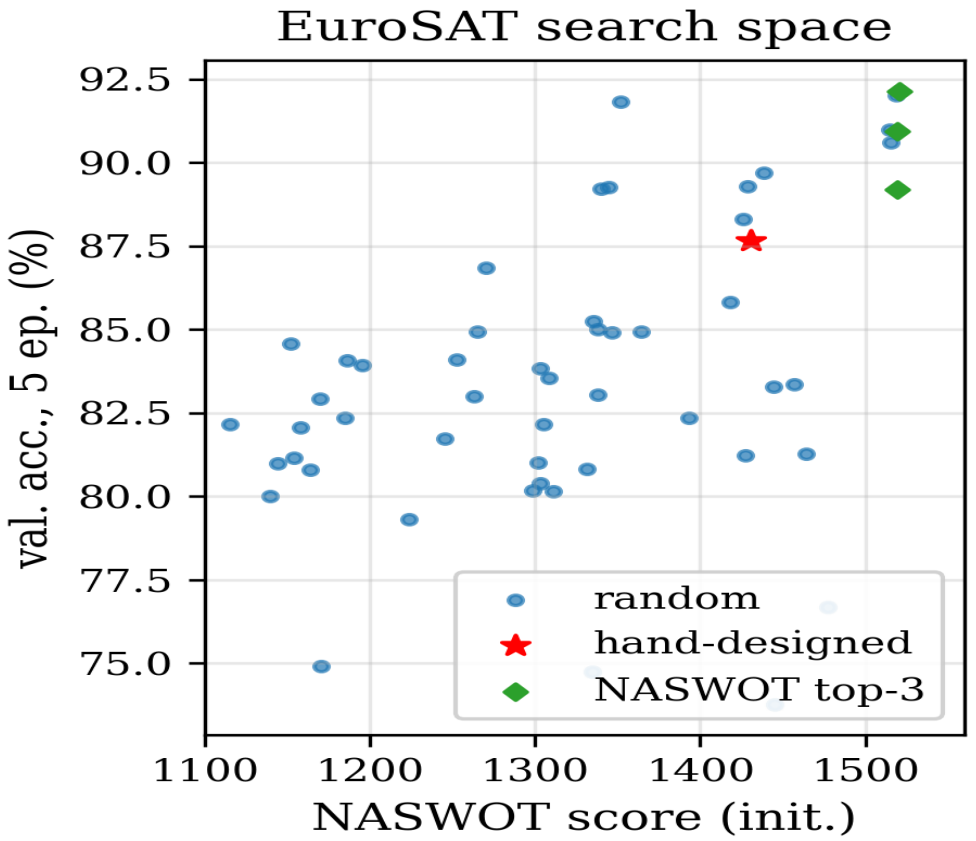
Step 2 — Zero-cost NAS on EuroSAT

- Search space = the Task 2 knobs: 2–5 blocks, kernel 3/5/7 **per block**, width 16/32/64, BN, pool/stride, skip → **8,640 CNNs**
- 51 networks trained 5 epochs to check the proxies (Spearman ρ):

	NASWOT	SynFlow	SNIP	grad-norm	#params
ρ	0.43	0.09	0.51	0.56	-0.13

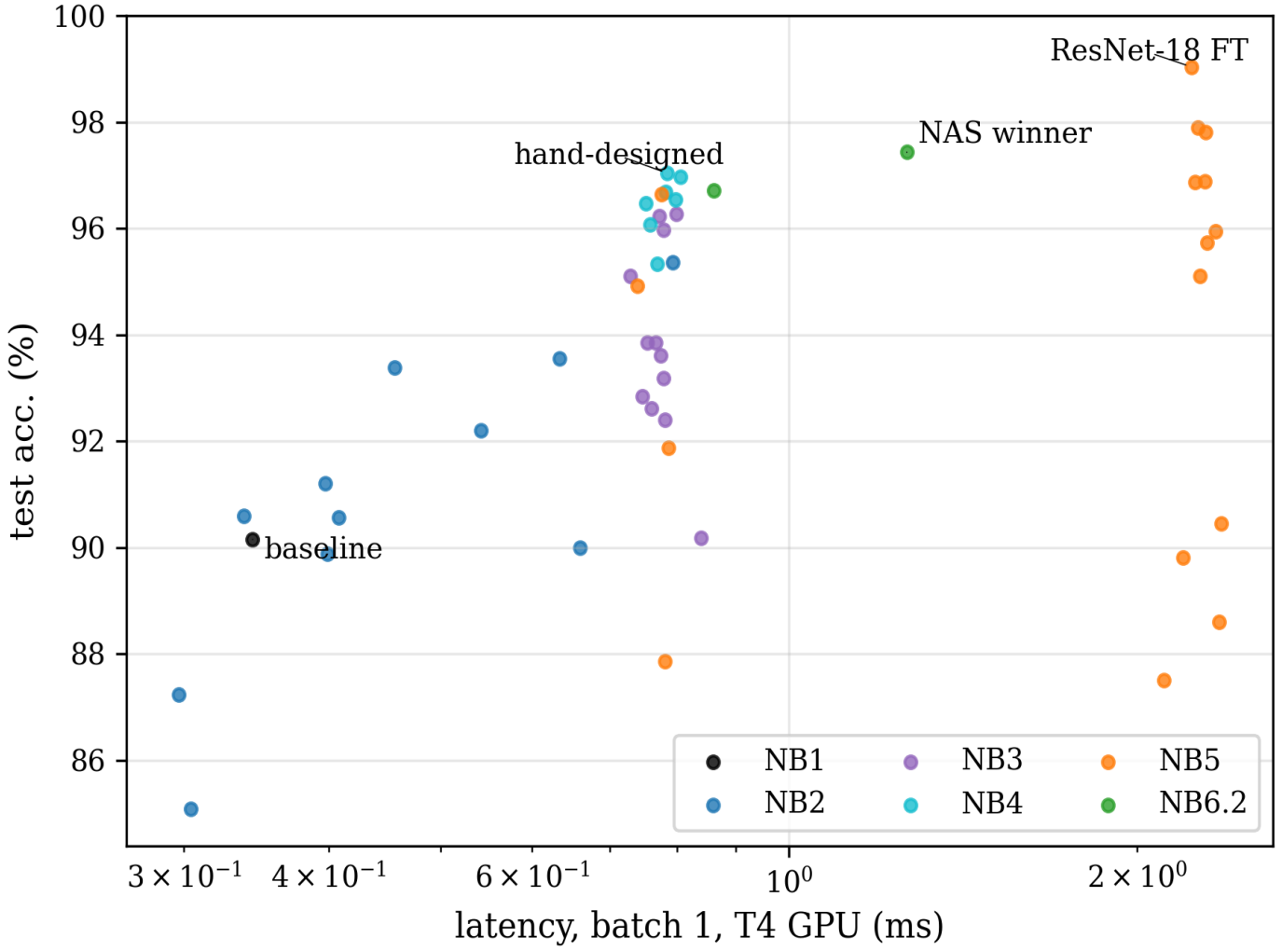
- Search: 1000 networks scored in 153 s, then the top-3 trained
- NASWOT top-3: **90.8%** mean (5 epochs) vs random 83.3%, hand-designed 87.7%

Full recipe	Test acc.	Params	Latency	Search GPU
Hand-designed	96.72%	1.24M	0.76 ms	203 min
NAS winner	**97.43%**	**1.99M**	**1.31 ms**	**17 min**



Winner: 5 blocks, kernels 5-3-3-3-3, width 64, BN, max-pool, **residual** → better than our design with **12× less GPU time**

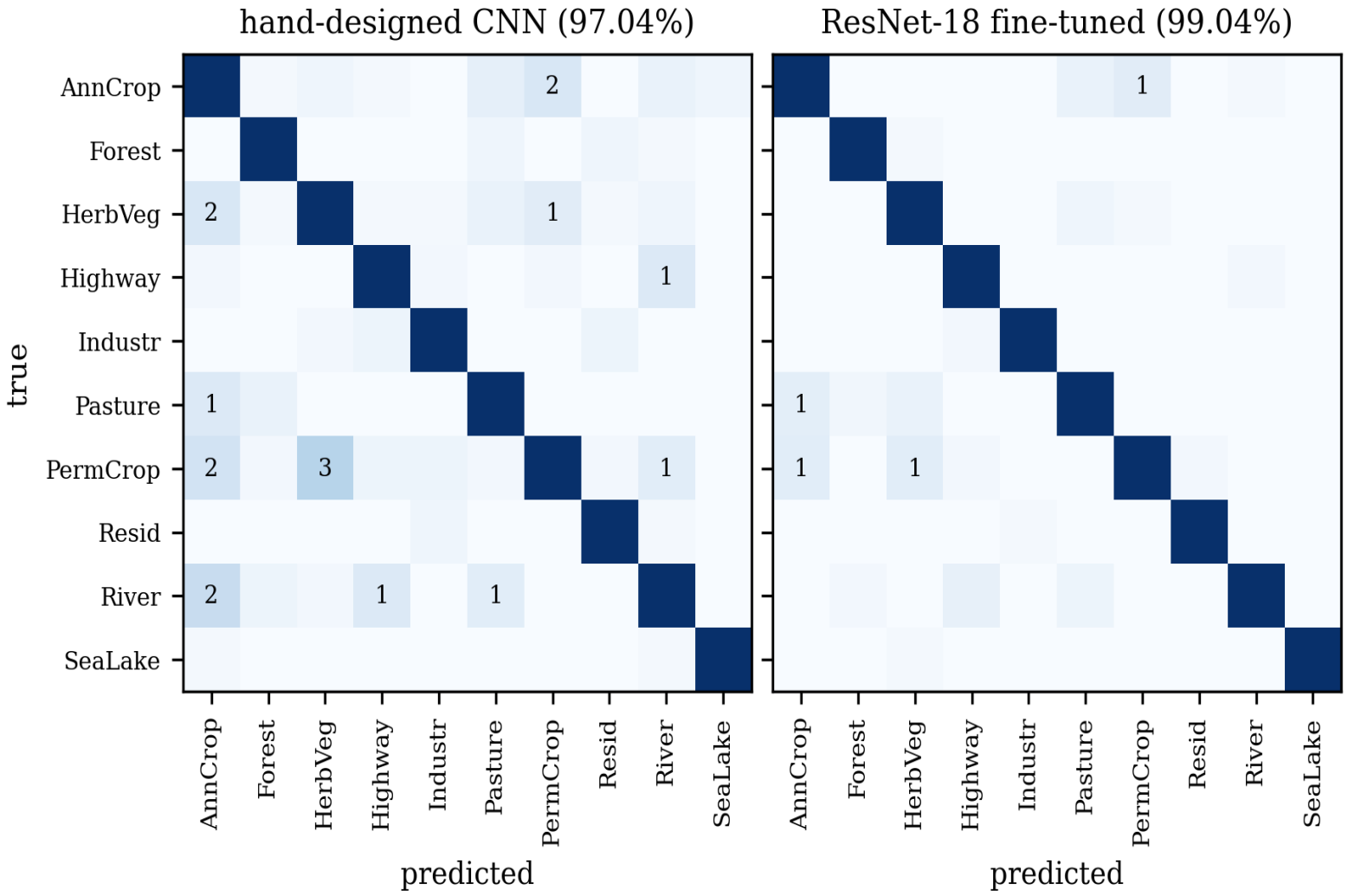
Final comparison — all 50 checkpoints (NB7)



Model	Test acc.	F1	Params	Latency
Baseline	90.15	89.75	2.19M	0.34 ms
Hand-designed	97.04	96.97	1.24M	0.79 ms
NAS winner	97.43	97.36	1.99M	1.26 ms
ResNet-18 FT	99.04	98.99	11.18M	2.23 ms

- Every checkpoint re-evaluated on the same 4,050 test images
- Trade-off: the NAS winner is 1.6× slower than the hand-designed CNN, ResNet-18 3× slower
- ResNet-18 is the most accurate, the scratch CNNs the fastest

Where do the models still fail?



120

errors — hand-designed CNN

39

errors — ResNet-18 fine-tuned

- Remaining errors: agricultural classes (PermanentCrop / HerbaceousVeg. / AnnualCrop / Pasture)
- ResNet makes **different** mistakes from the scratch CNNs
- 14 images wrong for all models: mixed land uses

Conclusions

90.15 → 97.04%

hand-tuning a CNN from scratch

99.04%

fine-tuned ResNet-18 (95.95% with 5% of the data)

97.43%

zero-cost NAS, 17 GPU-min vs 203

Findings

- Depth and small kernels, Adam with a decaying lr, orientation-invariant augmentation
- Transfer learning is the most accurate and by far the best with little data — at 9× the size
- Zero-cost NAS **reproduced** on NAS-Bench-201, then found a CNN **better than ours**

Limitations

- One seed per configuration; repeated runs vary by ≈0.3–1 pt → only large effects are conclusive
- EuroSAT NAS ground truth = a 5-epoch training
- ImageNet16 scored with substitute images
- **Live demo (NB7)**: load any checkpoint, classify an image, change lr / activation / loss / augmentation and retrain in seconds

Thank you — questions?

Notebooks NB0–NB7 · 50 checkpoints · report and figures generated from the saved results

Emilio Yared · Joe Missisian — Group 7